Implementation Report: Part 1 RTSSP and Part 2 SHP

Eléa-Rose Thomas 76156 Vicente Ribeiro 72990

May 13, 2026

1 Project Overview

The project is divided into two cooperating components. Part 1 focuses on encrypted media transport using RTSSP, while Part 2 extends the system with SHP, a secure handshake protocol used to authenticate the communicating parties and negotiate fresh cryptographic material before streaming starts.

In the RTSSP part, the server reads frames from a `.dat` movie file and sends them as encrypted UDP datagrams to a proxy. The proxy validates and decrypts the received packets, then forwards the recovered plaintext payload to a local client endpoint. This creates a secure transport path while preserving the real-time behaviour of the original stream.

The SHP part adds a setup phase before the media stream begins. During this phase, the proxy and server perform an authenticated ECDH-based handshake, negotiate a compatible ciphersuite, derive session keys, and confirm that both sides share the same cryptographic state. Once this is complete, RTSSP streaming begins using the negotiated session keys instead of static configuration keys.

The implementation is written entirely in Java and uses the standard JCE APIs for symmetric encryption, authenticated encryption, HMAC computation, ECDSA signatures, and ECDH key agreement.

2 Part 1: RTSSP Design Specification

2.1 Functional Goal

The goal of Part 1 is to provide an encrypted UDP streaming pipeline. The server reads movie frames in timestamp order, encrypts each frame payload, and sends the result to a proxy over UDP. The proxy then reverses the process by parsing the packet, checking its integrity when required, decrypting the payload, and forwarding the original frame data to a local delivery endpoint.

The protocol was designed to support multiple ciphersuites. In particular, it can use authenticated encryption modes such as AES-GCM and ChaCha20-Poly1305, as well as AES-CTR combined with a separate HMAC for integrity protection.

2.2 Packet Format

RTSSP packets are binary and self-describing. Each packet starts with the name of the configuration profile used to encrypt it, followed by the IV, ciphertext, and, when required, an HMAC.

```
uint8 config_name_length
uint8[] config_name
uint8[] iv
uint8[] ciphertext
uint8[] hmac (optional)
```

The configuration name is used by the receiver to select the correct cryptographic parameters from `Cryptoconfig.conf`. The IV length depends on the selected cipher mode. For AEAD modes, the authentication tag is already included in the ciphertext produced by the JCE provider. For CTR mode, the protocol appends an explicit HMAC to protect the packet header and ciphertext.

2.3 Cryptographic Design

The implementation supports three ciphersuites:

```
AES/GCM/NoPadding
CHACHA20-Poly1305
AES/CTR/NoPadding
```

The configuration parser accepts keys encoded in hexadecimal, Base64, or plain text. For AES-CTR, the configuration can also specify `HmacSHA256` together with a separate MAC key.

The integrity model depends on the selected ciphersuite. AES-GCM and ChaCha20-Poly1305 provide confidentiality and integrity through their built-in AEAD authentication tags. AES-CTR only provides confidentiality, so the implementation pairs it with HMAC-SHA256 to detect packet modification before decryption.

2.4 Implementation Details

Part 1 uses `hjStreamServer.java` as the streaming server. At startup, it parses `Cryptoconfig.conf` and loads the sections into a map.

The server then matches the selected movie file to the matching configuration section, for example `<movie>.encrypted`.

Each movie frame is read as a sequence containing a short payload size, a long timestamp, and the frame payload. For every frame, the server generates a fresh IV and encrypts only the payload. The frame boundaries are therefore represented by the UDP packet stream rather than by leaving frame metadata in plaintext. To preserve the original playback timing, the server sleeps between frames according to the timestamps stored in the movie file. During execution, it records runtime statistics in `hjStreamServer.stats.log`.

The proxy, implemented in `hjUDPproxy.java`, performs the complementary operation. It reads the remote and local endpoints from `config.properties`, listens for encrypted UDP datagrams, parses the packet header, and uses the configuration name to locate the matching cryptographic parameters. If an HMAC is present, it is verified before decryption. After successful decryption,

the proxy forwards the plaintext payload to one or more local delivery addresses. It also records packet drops, delivery failures, byte counters, and throughput in `hjUDPproxy.stats.log`.

2.5 Configuration

The configuration file is section-based, which makes it possible to support several movies and cryptographic profiles without changing the Java code. A typical section for an AES-CTR configuration is shown below.

```
<cars.dat.encrypted>
ciphersuite: AES/CTR/NoPadding
key: <hex key>
hmac: HmacSHA256
mackey: <hex key>
</cars.dat.encrypted>
```

This design keeps protocol logic separate from cryptographic parameters and makes testing different ciphersuites straightforward.

3 Part 2: SHP Design Specification

3.1 Functional Goal

Part 2 introduces SHP, a secure handshake protocol that runs before RTSSP streaming starts. Its purpose is to authenticate the client and server, bind the session to the requested movie, negotiate a ciphersuite, and derive fresh symmetric keys using ECDH.

After the handshake is complete, both parties confirm that they derived the same key material. Only then does the server begin sending RTSSP packets encrypted under the negotiated session keys.

3.2 Handshake Message Flow

SHP uses three binary messages: **ClientHello**, **ServerHello**, and **CSSP**. Each message begins with a message type and protocol version. Variable-length fields are encoded using a two-byte length followed by the corresponding byte array.

The proxy begins the handshake by sending a **ClientHello**. This message contains a self-signed EC certificate, an ephemeral ECDH public key, a random 32-byte nonce, the requested movie name, a list of supported ciphersuites, and an ECDSA signature over the message body.

The server receives this message and validates the client certificate and signature. It also checks that the requested movie matches the movie it is prepared to stream. The server then selects the strongest mutually supported ciphersuite, generates its own certificate and ephemeral ECDH keypair, signs its response, and sends back a **ServerHello**. This message also includes a response to the client's nonce challenge and a fresh server nonce.

Finally, the proxy sends a **CSSP** control message encrypted under the newly derived session keys. The plaintext includes the movie name, a response to the server's nonce challenge, and the command `START_RTSSP`. If the server successfully decrypts and validates this message, it begins RTSSP streaming with the negotiated session keys.

3.3 Key Establishment

The key establishment process is based on ephemeral ECDH over the `secp256r1` curve. Both sides compute the same shared secret from their private key and the peer's public key.

The implementation then derives the final session keys using HKDF-SHA256. The salt is generated from the client and server nonces, while the handshake transcript is included to bind the derived keys to the exact ClientHello and ServerHello messages exchanged during the session. HKDF expands this material into a 64-byte key block. This block is then split into an encryption key and, when the selected ciphersuite requires it, a separate HMAC key.

The default preference order for ciphersuite negotiation is:

```
AES/GCM/NoPadding
CHACHA20-Poly1305
AES/CTR/NoPadding
```

This means the implementation prefers AEAD encryption when possible, while still supporting AES-CTR with HMAC as a fallback.

3.4 Implementation Details

Most of the shared SHP logic is implemented in `SHPProtocol.java`. This class contains the helpers for EC keypair generation, certificate creation and verification, binary encoding and parsing, signed hello message construction, HKDF-based key derivation, CSSP encryption and verification, and ciphersuite negotiation.

The server-side implementation is provided by `hjStreamServerSHP.java`. It first completes the SHP handshake and then converts the resulting session keys into an RTSSP-style cryptographic configuration. From that point onward, it streams the movie in the same way as the Part 1 server, but with fresh session keys instead of static configuration keys.

The proxy-side implementation is provided by `hjUDPproxySHP.java`. It initiates the handshake, validates the server response, derives the same session keys, sends the encrypted CSSP message, and then receives, decrypts, and forwards RTSSP packets.

4 Testing and Observed Results

4.1 Integration Test Method

To test the full system in a reproducible way, we stream the given movie files for 40 frames. Each frame contained 64 bytes of payload, and consecutive timestamps were spaced 50 ms apart. A local UDP sink was bound to `localhost:7777` to verify that the proxy forwarded the decrypted traffic correctly.

The server and proxy were launched from their own working directories so that each component could read its expected configuration files and write its statistics logs in the correct location.

4.2 Part 1 Results

The Part 1 test completed successfully. The server log reported 40 segments read and sent, with no send failures and a 0% failure rate. The proxy log also reported 40 received segments,

Metric	Result
Forwarded UDP packets	40
Delivered payload bytes	2560
Server ciphersuite	AES/CTR/NoPadding
Observed stream duration	1.949 s
Observed payload throughput	10.51 kbps
Observed encrypted throughput	21.83 kbps

Table 1: Part 1 RTSSP test results

40 forwardable segments, and 40 delivered datagrams. No parsing errors, MAC failures, or decryption drops were observed.

These results show that the encrypted packet format, IV handling, HMAC verification, and proxy forwarding logic were consistent between the sender and receiver.

4.3 Part 2 Results

Metric	Result
Handshake completion time	1.002 s
Negotiated ciphersuite	AES/GCM/NoPadding
Forwarded UDP packets	40
Delivered payload bytes	2560
Observed stream duration	1.950 s
Observed payload throughput	10.50 kbps
Observed encrypted throughput	18.54 kbps

Table 2: Part 2 SHP + RTSSP test results

The Part 2 test also completed successfully. The SHP server log confirmed that the handshake finished before streaming began. The proxy log confirmed that the proxy initiated the handshake, validated the server response, derived the same session keys, and forwarded all RTSSP packets without errors.

The negotiated ciphersuite was `AES/GCM/NoPadding`, which matches the configured preference for AEAD encryption. As in Part 1, all 40 frames were delivered successfully and no parse, MAC, decryption, or delivery failures were recorded.

4.4 Evaluation of the Results

The test results indicate that both protocol layers behave as expected. Part 1 correctly encrypts and authenticates frame payloads, while the proxy is able to verify, decrypt, and forward the recovered plaintext. Part 2 successfully adds an authenticated setup phase before streaming and replaces static configuration keys with fresh keys derived from ECDH.

The packet counters from the server, proxy, and local UDP sink all match the expected 40-frame movie clips. This consistency is important because it shows that the packet format, key configuration, IV handling, and forwarding logic were aligned across the system.

5 Conclusion

The implementation successfully delivers the two required protocol layers. RTSSP provides a configurable encrypted UDP streaming path with support for both AEAD encryption and AES-CTR with HMAC. SHP extends this design by adding an authenticated handshake that negotiates cryptographic parameters and derives fresh per-session keys before streaming begins.

The final integration tests compiled and ran successfully. In both parts, the system delivered all 40 expected frames, recorded no packet drops, and produced consistent statistics across the server and proxy logs. These results show that the implemented design is functional, interoperable between the server and proxy, and suitable for the secure streaming scenario defined in the project.